Les règles d'équipe

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Introduction 3

II. Respect des principes et des valeurs 3

A. Comment interpréter les principes et les valeurs ? 3

III. Développer ses compétences 5

IV. Definition of ready (DoR) 5

V. Definition of done (DoD) 5

VI. Auto-évaluation 6

Solutions des exercices 7

Tous droits réservés à STUDI - Reproduction interdite

I. Introduction

Des règles pour encadrer le travail d'équipe

Dans ce chapitre, nous allons revenir sur les différents éléments de Scrum abordés dans les premières parties du cours, mais en détaillant comment ces éléments peuvent aider ou guider les développeurs dans la réalisation du produit.

Parmi les différentes manières d'encadrer le travail des équipes, nous allons voir que certains principes et valeurs sont appliqués comme des règles. Il y a d'une part les definitions of ready et les definitions of done qui sont des contrats passés entre les développeurs, et d'autre part les analystes métier (dont le PO) et les testeurs. Ces groupes forment 2 ensembles distincts de règles.

Enfin, nous nous pencherons sur les règles d'équipes « customisées » qui sont initiées par un ou plusieurs membres de l'équipe Scrum, puis validées par tous.

II. Respect des principes et des valeurs

A. Comment interpréter les principes et les valeurs ?

Revenons sur les 4 valeurs défendues dans le Manifeste Agile[1] et sur ce qu'elles impliquent pour les développeurs :

- « ***Les individus et leurs interactions*** » **plus que** « ***les processus et les outils*** » **:** la seconde partie de la phrase n'est pas vitale pour les développeurs en dehors des outils de développement ; ils peuvent donc être moins vigilants sur celle-ci. Il est donc important que les développeurs comprennent que des processus qui ne sont pas toujours suivis et des outils qui ne sont parfois pas à jour vont générer du flux dans l'équipe, ce qui peut faire baisser la productivité. Pour que le PO anticipe les besoins de l'équipe, il a besoin d'une vision claire. Mais il est préférable de se baser sur l'interaction avec les autres plutôt que de compter sur une utilisation parfaite des processus et des outils.
- « ***Des produits opérationnels*** » **plus qu'** « ***une documentation exhaustive*** » **:** on pourrait penser que cela signifie que la documentation est l'ennemi du produit, mais ce n'est pas le cas. Un produit de qualité et qui fonctionne est la priorité. Cependant, il faut de la documentation pour maintenir une compréhension des bonnes pratiques en vigueur sur le projet, une bonne connaissance de la finalité et du fonctionnement des fonctionnalités ainsi qu'une bonne compréhension de la solution technique. Le mieux est de discuter au début du projet de la documentation nécessaire et d'y revenir régulièrement. À noter que la documentation peut être intégrée au code pour éviter la dispersion d'informations.
- « ***La collaboration avec les clients*** » **plus que** « ***la négociation contractuelle*** » **:** pour le développeur, cela induit une certaine liberté dans les échanges au cours de la sprint review : les développeurs présentent les fonctionnalités qu'ils ont développées directement au client. Il leur pose des questions auxquelles ils répondent directement. Le contrat est donc nécessaire pour convenir des obligations légales.
- « ***L'adaptation au changement*** » **plus que** « ***le suivi d'un plan*** » **:** avec l'Agile, on cherche à s'adapter à la réalité de la situation et du besoin en permanence. Pour autant, on ne néglige pas les plans qui restent incontournables pour se projeter et avoir un minimum de certitude. Ce qui change avec l'Agile, c'est que le plan n'est pas le plus important et qu'il faut rester vigilant pour voir quand il faut le suivre et surtout ne plus le suivre. Le niveau de précision du plan peut également changer pour rester plus simple à suivre.

Viennent ensuite les principes de l'Agile. Nous retiendrons les principaux :

- **Satisfaire le client en délivrant de la valeur tôt et régulièrement :** il est important pour le développeur de prendre en compte la satisfaction client. Traditionnellement, les développeurs sont centrés sur ce que le responsable technique leur donne à faire : on satisfait la demande du tech lead. En Agile cependant, le développeur est fortement sensibilisé à la façon dont le produit doit servir le besoin du client, ce qui change la façon d'envisager une fonctionnalité : cela passe ainsi plus souvent par une conception collégiale.

[1] https://agilemanifesto.org/iso/fr/manifesto.html

Tous droits réservés à STUDI - Reproduction interdite

- **Construire des projets autour de personnes motivées :** ceci a son importance car ceux qui construisent le plus le produit sont les développeurs. Et la motivation est cruciale pour s'engager sur le contenu d'un sprint et pour s'intéresser aux besoins du client. Si vous n'êtes pas motivé par un projet, posez-vous la question : pourquoi ? Il y aura toujours quelque chose de factuel à exposer en rétrospective pour que le projet soit plus passionnant. Cette démarche relève de l'auto-organisation.
- **Privilégier le face-à-face :** l'échange direct avec quelqu'un est en effet plus efficace. Cela s'applique aux développeurs qui sont souvent amenés à communiquer en pair programming pour de la conception ou pour discuter des US avec le métier et les autres membres de l'équipe. Lorsque le télétravail est inévitable, il faut s'efforcer d'utiliser tous les moyens technologiques disponibles et de bien cerner son besoin d'échange avant d'organiser ses plages de travail et ses plages de disponibilité, quitte à mettre en place un système de flags pour en informer les autres (sans oublier de garder des moments de convivialité malgré la distance).
- **Porter une attention continue à l'excellence de la réalisation :** c'est un principe qui concerne essentiellement les développeurs. Prendre conscience de l'impact de la dette technique sur la qualité d'un produit est d'autant plus important que le produit est construit par petits incréments. L'ambition de l'Agile est de corriger en permanence le code pour que les malfaçons n'engendrent pas des dysfonctionnements importants voire irrattrapables. Les itérations sont autant d'occasions d'en apprendre plus sur la qualité du code et les outils d'analyse qui sont de précieux alliés.
- **Privilégier la simplicité :** trop d'anticipation du développeur peut complexifier les flux sans apporter de valeur au produit ou au client. Il est donc important d'attendre la sprint review pour connaître la direction que prend le produit, la satisfaction du client et s'interroger sur la pertinence de l'ajout imaginé. Par ailleurs, faire simple permet d'aller plus vite et ainsi d'être plus efficace.
- **Les équipes auto-organisées sont les plus performantes :** quand on est auto-organisé, on sait ce qu'on doit faire, où, et quand : on est plus efficace. Pour le développeur, cela ne signifie pas qu'il sait tout ou doit tout trouver lui-même. Il suit d'abord les consignes du Scrum master et discute de ses questions ou suggestions lors de la rétrospective.
- **Réfléchir à ses pratiques et les ajuster régulièrement :** ce n'est pas toujours évident pour les développeurs, car le développement nécessite d'être plongé dans le code et de chercher des solutions, ce qui ne facilite pas la prise de recul nécessaire pour réfléchir à ses pratiques. Il faut d'abord se rappeler que l'Agile préconise la bienveillance dans le partage de ses difficultés ou incompréhensions. Le moment le plus approprié pour en faire part est la rétrospective, bien que cela puisse se faire avant. En effet, afin d'être compris par les autres, il est important de s'en tenir aux faits, en évitant les jugements et les argumentaires inutiles. Ensuite, il s'agit de faire confiance aux collègues pour proposer des solutions et au Scrum master pour rappeler les règles – notamment de bienveillance – en cas de manquement. Enfin, il faut se montrer disponible aux propositions du Scrum master pour chercher des solutions via des exercices ou des ateliers.

Enfin, voici comment les valeurs *Scrum tirées du Guide* s'appliquent aux développeurs :

- **L'engagement :** pour le développeur, il s'agit de respecter ce qu'il a validé en Sprint Planning, en faisant le maximum pour atteindre les objectifs donnés. Le principe Agile « *l'effort doit être durable* » doit venir modérer l'ambition de l'équipe ou du product owner si nécessaire. Cela signifie s'engager dans les actions d'amélioration décidées en rétrospective.
- **La concentration :** chacun doit être focus sur les objectifs de l'équipe et ne pas se laisser distraire par des demandes venant de l'extérieur de l'équipe, pour rendre service, par exemple. Si besoin, le Scrum master est là pour protéger des distractions.
- **L'ouverture :** Scrum suppose une ouverture au travail et aux challenges. Quel que soit le travail à faire pour atteindre les objectifs ou les challenges à relever, il faut rester ouvert. Cela signifie ne pas refuser une tâche parce que ce n'est pas ce qu'on aime faire ou ne pas refuser un challenge parce qu'il est difficile, mais plutôt chercher à le surmonter.
- **Le respect :** chacun dans l'équipe respecte les autres, leur indépendance et leur capacité à faire. Chaque développeur doit considérer l'autre comme capable et ne pas remettre en cause unilatéralement son choix de solution.

Tous droits réservés à STUDI - Reproduction interdite

- **Le courage :** pour le développeur, cela signifie être ambitieux dans ses estimations de développement mais aussi ne pas céder à la facilité.

III. Développer ses compétences

Comment et pourquoi développer ses compétences ?

L'équipe Scrum est constituée de telle sorte qu'elle réunit toutes les compétences nécessaires à la réalisation du produit. Pour autant, développer de nouvelles compétences facilite l'équilibrage de la charge de travail sur l'ensemble de l'équipe et augmente la capacité de l'équipe.

Toutefois, il faut rester concentré sur les objectifs du projet et améliorer ses compétences au profit de celui-ci. Chaque développeur a généralement 1 ou 2 compétences fortes ou domaines d'expertise. Développer des compétences secondaires peut permettre de suppléer un collègue sur un sujet et ainsi contribuer plus fortement à l'atteinte des objectifs.

Quelles compétences secondaires choisir ?

Lorsqu'il s'agit de développer des compétences secondaires, 2 grandes possibilités existent pour le développeur :

- Première option : Un développeur « Front Office » peut développer une compétence sur le « Back Office » et ainsi coder une fonctionnalité intégralement.
- Seconde option : Il peut aussi développer une compétence en test ou en rédaction d'US, ce qui permet de fluidifier le processus, en amont ou en aval du développement.

À noter : ce développement des compétences est une façon d'appliquer les valeurs d'engagement, et d'ouverture (au travail et aux challenges) vues précédemment.

IV. Definition of ready (DoR)

Qu'est-ce que la DoR ?

La definition of ready concerne le product owner et ses aides, mais son application impacte essentiellement le développeur.

Comme cette définition se fait par consensus, le développeur a sa part de responsabilité. Son devoir est d'expliquer son besoin dans la rédaction des user stories en précisant les éléments qu'il ne pourra pas définir simplement par lui-même. La rédaction d'une user story (US) doit permettre de réaliser le développement sans difficulté particulière. Les développeurs doivent pouvoir « dérouler » facilement le codage afin que les estimations soient relativement précises et sûres au moment de la planification du sprint, et qu'une certaine prédictibilité soit possible au niveau de la feuille de route définissant les versions du produit.

Si la difficulté est inévitable, il est conseillé de la gérer via un pic technique (ou pike), dont l'objectif sera d'explorer les solutions possibles et d'en choisir une pour fiabiliser l'estimation de la user story, en éliminant la composante aléatoire. Dans le cas contraire, il sera nécessaire de prendre une marge importante, ce qui provoquera un flou sur l'atteinte des objectifs du sprint.

Comme expliqué précédemment, le développeur peut aider le product owner pour que la user story respecte la DoR en développant des compétences secondaires dans la rédaction des user stories.

V. Definition of done (DoD)

Quand peut-on livrer un développement ?

La definition of done est appliquée par les développeurs et négociée par les mêmes parties que la definition of ready. Elle correspond également à un ensemble de règles qui se traduisent par des tâches à faire figurer sur le task board.

Tous droits réservés à STUDI - Reproduction interdite

Cette définition doit s'adapter à la situation : inutile d'être excessivement exigeant si c'est pour générer de la dette technique ou fausser la qualité du code par des tests non faits. Il s'agit d'établir un contrat entre le Product owner et les développeurs pour savoir ce qui est fait quand une user story est « done ».

Généralement, l'application rigoureuse de la definition of done nécessite discipline et courage.

Enfin, plus concrètement, la définition est basée sur un accord concernant la documentation nécessaire, l'application des règles de codage, des règles liées à la gestion de configuration ainsi que les tâches à réaliser pour l'intégration continue. Les règles de codage sont explicites, donc très faciles à vérifier par l'analyse de code.

Exemple **Des règles concrètes**

Voici quelques exemples de règles concrètes pouvant émerger de la definition of done :

- Mettre à jour le manuel utilisateur.
- Définir l'objectif d'une fonction dans les commentaires et lister les exceptions.
- « Merger » si les tests unitaires sont OK, si aucune erreur n'est remontée par l'analyse de code et si toutes les alertes sont prises en compte ou motivées.
- Tests croisés réalisés et OK non bloquants.

VI. Auto-évaluation

Exercice : Quiz

Question 1

La montée en compétence est une façon de mettre en œuvre les valeurs suivantes (plusieurs réponses possibles) :

- ☐ Engagement
- ☐ Respect
- ☐ Ouverture
- ☐ Focus

Question 2

La definition of ready ne permet pas :

- ○ De fluidifier le flux de travail
- ○ D'indiquer aux développeurs qu'une story peut être développée
- ○ D'améliorer la fiabilité des estimations
- ○ D'avoir un minimum de garantie sur la réalisation de la roadmap

Question 3

La definition of done donne une indication de sa capacité à être intégrée.

- ○ Vrai
- ○ Faux

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Un développeur doit faire simple, car (plusieurs réponses possibles) :

- ☐ Le code inutile complexifie le code
- ☐ Le code « *en plus* » aurait une valeur nulle
- ☐ C'est une façon d'appliquer la valeur de respect évoquée dans le guide Scrum
- ☐ Cela augmente sa productivité et donc celle de l'équipe
- ☐ Il faut respecter les règles

Question 5

Être courageux en Agile, c'est (plusieurs réponses possibles) :

- ☐ S'engager au plus juste sur les estimations en sprint planning
- ☐ Oser prendre la parole devant le groupe
- ☐ Oser faire les bonnes choses, et ne pas s'arrêter sur des solutions moyennes, mais coûteuses
- ☐ Refuser le travail quand il ne correspond pas à nos compétences

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 6 Solution n°1

Question 1

La montée en compétence est une façon de mettre en œuvre les valeurs suivantes (plusieurs réponses possibles) :

- [x] Engagement
- [ ] Respect
- [x] Ouverture
- [ ] Focus

Acquérir des compétences est une façon de s'engager dans l'atteinte des objectifs et est une marque d'ouverture vis-à-vis du travail à effectuer par l'équipe.

Question 2

La definition of ready ne permet pas :

- ○ De fluidifier le flux de travail
- ◉ D'indiquer aux développeurs qu'une story peut être développée
- ○ D'améliorer la fiabilité des estimations
- ○ D'avoir un minimum de garantie sur la réalisation de la roadmap

Une story « *ready* » est prête à être développée. Mais pour qu'elle le soit réellement, il faut qu'elle soit planifiée dans le sprint par l'équipe : ce n'est pas l'indicateur DoR qui l'inclut automatiquement dans le planning.

Question 3

La definition of done donne une indication de sa capacité à être intégrée.

- ◉ Vrai
- ○ Faux

Il est nécessaire que les règles relatives à l'intégration soient réalisées, mais ce n'est pas suffisant.

Question 4

Un développeur doit faire simple, car (plusieurs réponses possibles) :

- [x] Le code inutile complexifie le code
- [x] Le code « *en plus* » aurait une valeur nulle
- [ ] C'est une façon d'appliquer la valeur de respect évoquée dans le guide Scrum
- [x] Cela augmente sa productivité et donc celle de l'équipe
- [ ] Il faut respecter les règles

Prévoir un besoin non demandé par le client n'ajoute pas de valeur. Le code ne correspondant pas à une demande devient du code « *mort* », jamais utilisé, qui pollue le code utile et rend difficile la compréhension des objets concernés. En ne gâchant pas son temps sur des développements sans valeur, le développeur se concentre sur du code qui sert les objectifs du sprint.

Question 5

Tous droits réservés à STUDI - Reproduction interdite

Être courageux en Agile, c'est (plusieurs réponses possibles) :

- [x] S'engager au plus juste sur les estimations en sprint planning
- [ ] Oser prendre la parole devant le groupe
- [x] Oser faire les bonnes choses, et ne pas s'arrêter sur des solutions moyennes, mais coûteuses
- [ ] Refuser le travail quand il ne correspond pas à nos compétences

Refuser un travail n'est pas en phase avec la valeur d'ouverture préconisée par Scrum qui consiste à accepter le travail qui est à faire. Toutefois, la discussion reste possible si des éléments paraissent plus urgents et adaptés à nos compétences.

Tous droits réservés à STUDI - Reproduction interdite